

Computer Graphics (CACS305)

► Unit - 07: Visible Surface Determination & Computer Graphics Algorithm

Compiled by Shishir Ghimire

Credit Hours: 3hrs

► TABLE OF CONTENTS

Unit 7: Visible Surface Detections (5 Hrs.)

7.1 Image Space and Object Space Techniques

7.2 Back Face Detection, Depth Buffer (Z-buffer), A-Buffer and Scan-Line Algorithms.

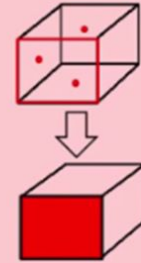
7.3 Depth Sorting Method (Painter's Algorithm)

7.4 BSP tree Method, Octree and Ray Tracing

► Introduction ► Terms:

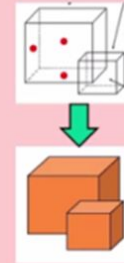
► Visibility Culling

Refers to the detection of which surfaces are completely Visible with respect to the camera.



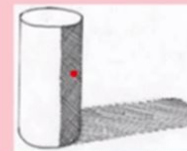
► Hidden-Surface Elimination

Refers to the elimination of the parts of the surfaces which are hidden with respect to the camera.



► Shadows

Refer to the determination of the parts of the surfaces which are hidden with respect to the light sources



The background features a dark blue gradient with various white and light blue geometric elements. These include thin lines, circles, and rectangles of different sizes and orientations, some of which are connected by lines, creating a technical or circuit-like aesthetic. A prominent light blue triangle points towards the main title text.

Image Space & Object Space Techniques and their Comparison

7.1

► Visible Surface Detection Techniques (Hidden Surface Removal):

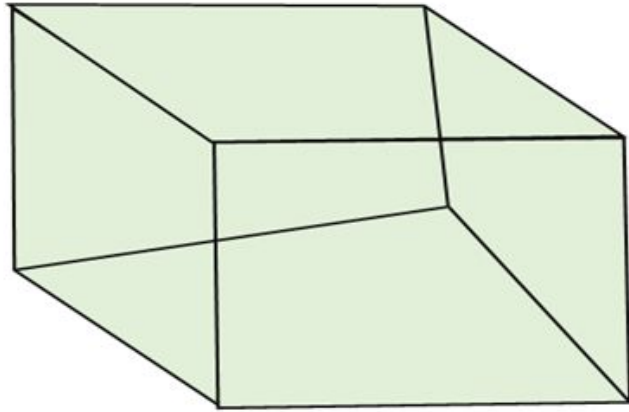
Problem: One of the most challenging problems in computer graphics is the **removal of hidden parts** from images of solid objects.

- In real life, the **opaque material** of these objects **obstructs the light rays** from hidden parts and prevents us from seeing them.
- In the computer generation, **no such automatic elimination** takes place when objects are projected onto the screen coordinate system.
- Instead, all parts of every object, including many parts that **should be invisible** are displayed.
- To remove these parts to **create a more realistic image**, we must apply a hidden line or hidden surface algorithm to set of objects.

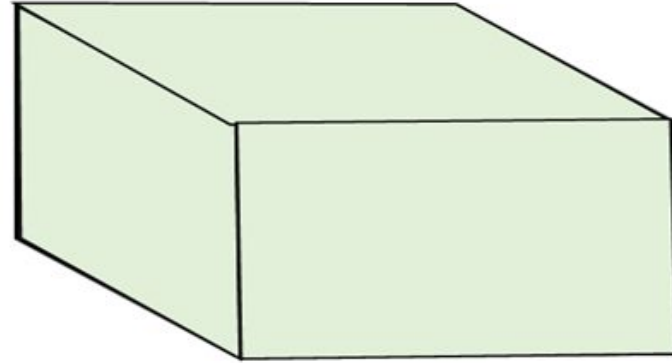
Definition: Hidden surface removal or **visible surface determination** is the process used to determine which surfaces and parts of surfaces are not visible from a certain viewpoint.

- Hidden surface determination is necessary to render an image correctly, so that one **may not view features hidden** behind the model itself, allowing only the naturally viewable portion of the graphics to be visible.
- Visible surface detection methods are broadly classified according to whether they deal with objects or with their projected images.

► Hidden Surface Removal - Comparison:



Object with hidden line

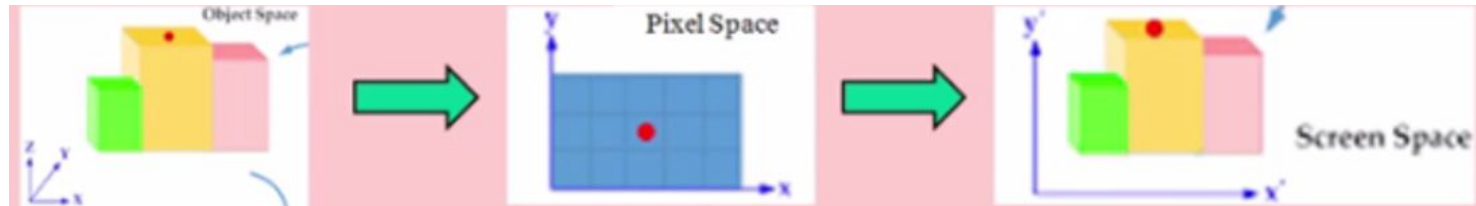
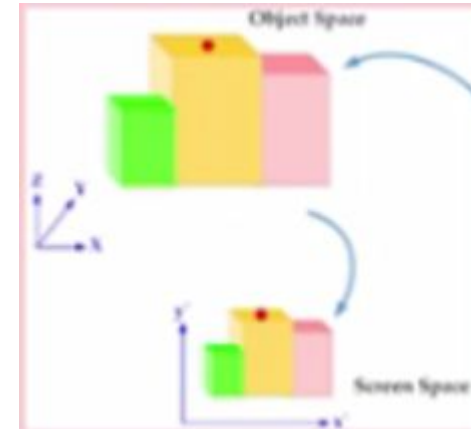


Object when hidden lines removed

► Hidden Surface Removal Approaches :

These two approaches are:

1. Object Space Methods
2. Image Space Methods



► Object Space Methods:

- ❖ **Overview:** These methods involve **comparing** different **parts of objects** to determine which surface are visible.
- ❖ **Mechanism:** After comparison, **visible, invisible, or partially** visible surfaces are identified.
- ❖ It is suitable for scene with small number of objects and objects with simple geometric descriptions.
- ❖ **Application:** Commonly used in **wireframe models** to determine visible lines, making these algorithms **line-based** rather than surface-based.
- ❖ It is easy to implement but slower than other methods.

► Object Space Methods:

Basic Procedure:

For each object in the scene do:

1. Begin
 - a. Determine those parts of the object whose view is unobstructed by other parts of it or any object with respect to viewing specification.
 - b. Draw those parts in the object color
2. End

► Object Space Methods > Characteristics:

- ❖ **Compare each object with other objects** to determine the visibility of the object parts.
- ❖ If there are n objects then complexity is $O(n^2)$.
- ❖ Calculations are performed at the **resolution** in which the objects are defined.
- ❖ **Display is more accurate** but computationally more expensive as compared to image space methods due to the possibility of intersection between the surfaces.
- ❖ Suitable with **smaller number of objects** and objects with simple relationship with each other.

► Object Space Methods :

Popular Object Space Methods:

1. Back-Face Removal
2. Depth Sorting (Painter Algorithm)
3. Area Subdivision
4. Binary Space Partition Trees

▶ 2. Image Space Methods

- ❖ **Overview:** Visibility is decided point by point **at each pixel position** on the projection plane.
- ❖ **Mechanism:** The visibility of each pixel is assessed **individually**.
- ❖ **Application:** Used to locate visible surfaces **instead of** visible lines, making these methods **surface-based**.
- ❖ Most visible surface detection algorithm use image-space-method but in some cases object space methods are also used for it.
- ❖ Color calculation in **object space method** is only done once and is retained but in **image space method** the calculation is done once but overwritten later.

► Image Space Methods:

Basic Procedure:

For Each pixel in the image do:

1. Begin
 - a. Determine the pixel closest to that pixel
 - b. Draw the pixel in the object color
2. End

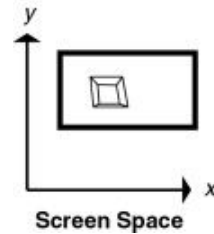
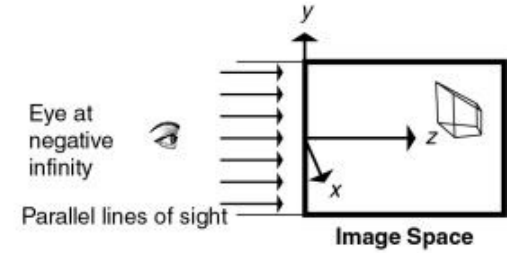
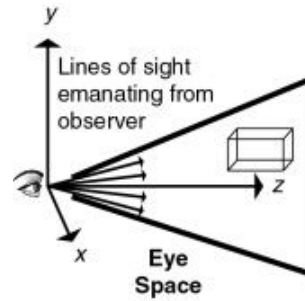
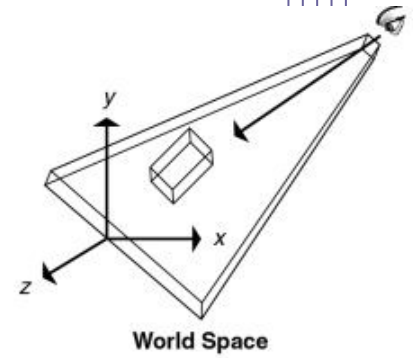
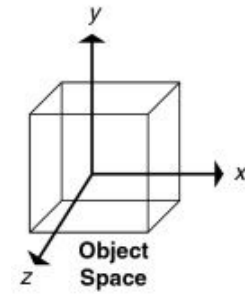
► Image Space Methods ► Characteristics:

- ❖ For each pixel, examine all the n objects to determine one closest to the viewer.
- ❖ If there are p pixels in the image, then complexity is $O(n*p)$.
- ❖ Accuracy of the calculation is bounded by the display resolution.
- ❖ A change in display resolution requires re-calculation.

► Image Space Methods :

Popular Image Space Methods:

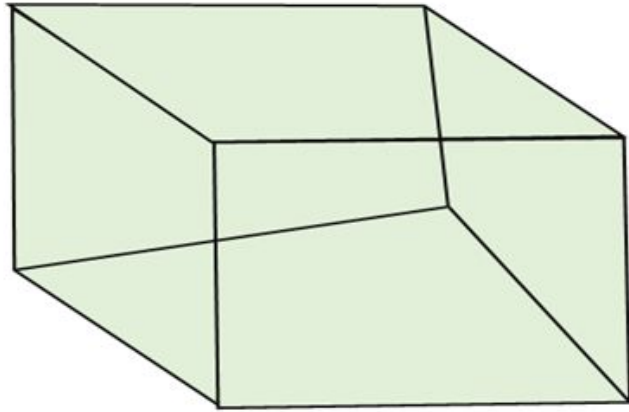
1. Depth Buffer (Z-Buffer)
2. Scan Line
3. Depth Sorting (Painter's Algorithm)



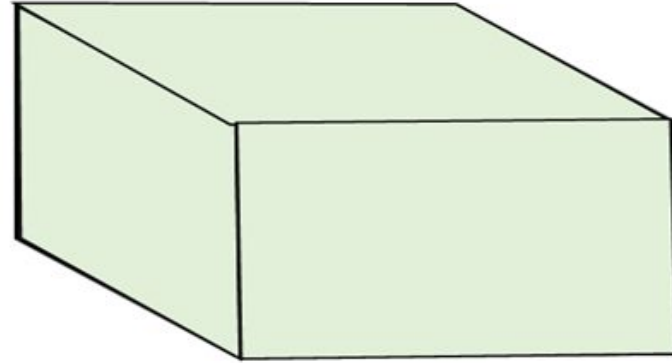
► Image Space vs Object Space Comparison:

Object Space	Image Space
1. Image space is object based. It concentrates on geometrical relation among objects in the scene.	1. It is a pixel-based method. It is concerned with the final image, what is visible within each raster pixel.
2. Here surface visibility is determined.	2. Here line visibility or point visibility is determined.
3. It is performed at the precision with which each object is defined, No resolution is considered.	3. It is performed using the resolution of the display device.
4. Calculations are not based on the resolution of the display so change of object can be easily adjusted.	4. Calculations are resolution base, so the change is difficult to adjust.
5. These were developed for vector graphics system.	5. These are developed for raster devices.
6. Object-based algorithms operate on continuous object data.	6. These operate on object data.
7. Vector display used for object method has large address space.	7. Raster systems used for image space methods have limited address space.
8. Object precision is used for application where speed is required.	8. There are suitable for application where accuracy is required.
9. It requires a lot of calculations if the image is to enlarge.	9. Image can be enlarged without losing accuracy.
10. If the number of objects in the scene increases, computation time also increases.	10. In this method complexity increase with the complexity of visible parts.

► Hidden Surface removal - Comparison:



Object with hidden line



Object when hidden lines removed

Back Face ► Removal Algorithm

7.2.
1

► Back Face Removal (Plane Equation

Method):

A fast and simple **object space method** used to **remove hidden surface** from a 3D object drawing is known as "Plane equation method" and applied to each side after any rotation of the object takes place.

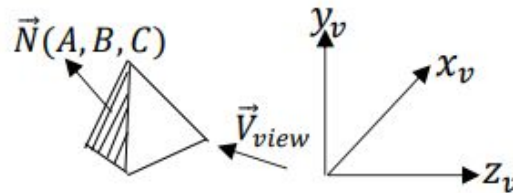
Purpose:

- To eliminate the rendering of faces (polygons) of a 3D object that are facing away from the viewer.
- Reduces the number of polygons that need to be processed and displayed.

A point (x, y, z) is inside a surface with plane parameter A, B, C & D if $Ax + By + Cz + D < 0$
i.e. it is in back face.

Let $\vec{N}$ be the normal vector for a polygon and $\vec{V}_{view}$ be the vector in viewing direction, then a polygon surface is back if

$$\vec{V}_{view} \cdot \vec{N} > 0$$

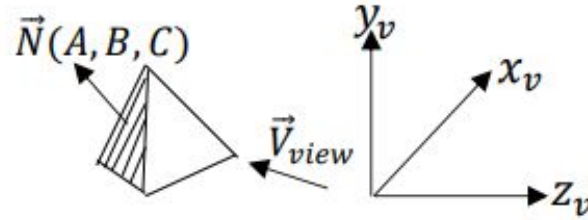


► Back Face Removal (Plane Equation

Method):

Let $\vec{N}$ be the normal vector for a polygon and $\vec{V}_{view}$ be the vector in viewing direction, then a polygon surface is back if

$$\vec{V}_{view} \cdot \vec{N} > 0$$



- For the **left handed viewing system** if the 'z' component of the normal vector is positive, then it is back face. If the 'z' component of the vector is negative then it is a front face.
- For the **right handed viewing system** if the 'z' component of the normal vector is negative, then it is back face. If the 'z' component of the vector is positive then it is a front face.

► Algorithm:

Repeat for all polygons in the scene.

1. Do numbering of all polygons in clockwise direction i.e.

$$v_1 v_2 v_3 \dots v_z$$

2. Calculate normal vector i.e. N_1

$$N_1 = (v_2 - v_1) \times (v_3 - v_1)$$

3. Consider projector P, it is projection from any vertex

Calculate dot product

$$\text{Dot} = N \cdot P$$

4. Test and plot whether the surface is visible or not.

If $\text{Dot} \geq 0$ then

surface is visible

else

Not visible

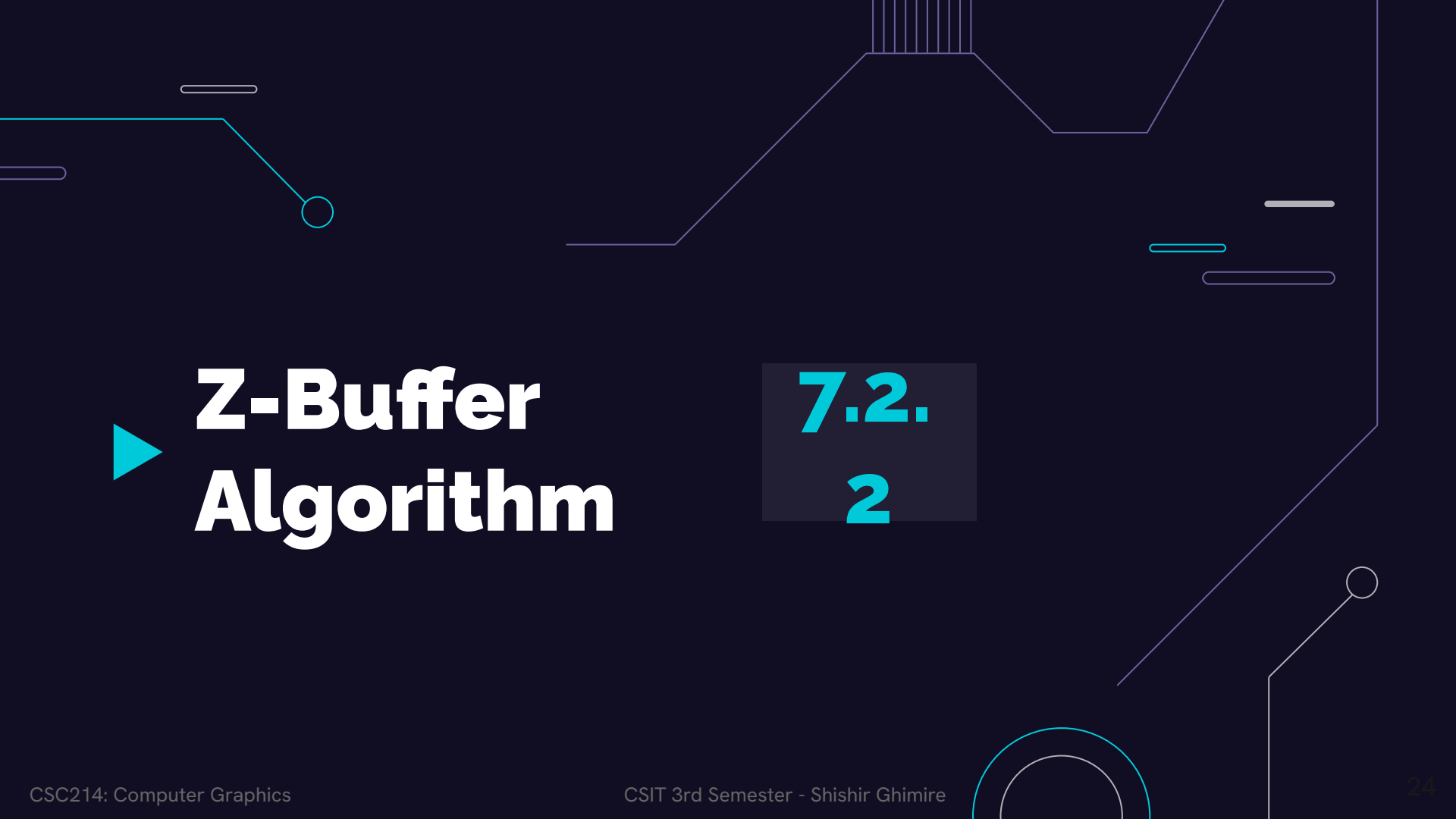
► Back Face Removal (Plane Equation Method):

Advantages:

- **Efficiency:** Reduces the number of polygons that need to be **processed and drawn**, which enhances rendering speed.
- **Simplicity:** Easy to implement and integrate into rendering pipelines.

Limitations:

- This method works fine for convex polyhedral, but not necessarily for **concave polyhedral or overlapping objects**. So, we need to apply other methods to further determine where the obscured faces are partially or completely hidden by other objects (e.g. Using Depth-Buffer Method or Depth-sort Method).
- This method can only be used on **solid objects** modeled as a polygon mesh.



► Z-Buffer Algorithm

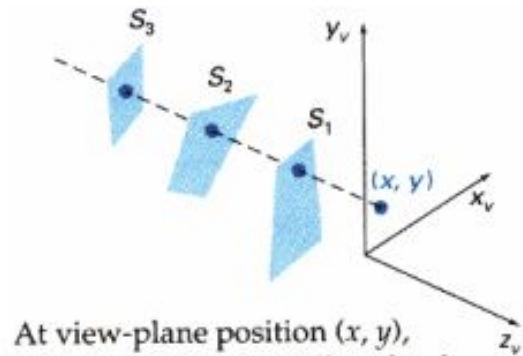
7.2.
2

► Z-Buffer Algorithm (Depth-Buffer):

- ❖ It is an **image space method** for detecting visible surface.
- ❖ The Z-buffer method compares **surface depths of each pixel position** on the projection plane. It is called z-buffer method since object depth is usually measured from the **view plane along the z-axis** of a viewing system.
- ❖ Normally z-axis is represented as **depth**.
- ❖ In this method, two buffers are used:
 - **Depth Buffer:** Stores depth values for each (x, y) position.
 - **Frame Buffer:** Stores the intensity values for each position.
- ❖ Each surface of scene is processed separately, **one point at a time** across the surface. The method is usually applied to scenes containing **only polygon surfaces**, because depth values can be computed very quickly and method is easy to implement. This method can be applied also to non planar surfaces.

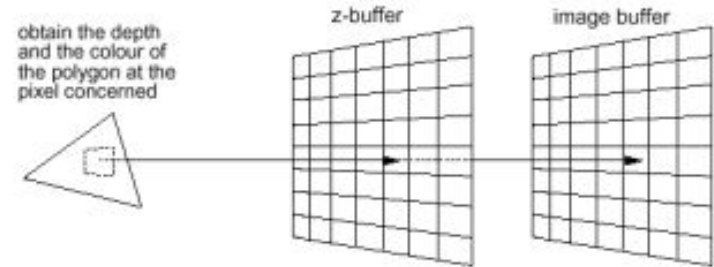
► Z-Buffer Algorithm (Depth-Buffer):

With object description converted to projection co-ordinates, each (x, y, z) position on polygon surface corresponds to the orthographic projection point (x, y) on the view plane. Therefore for each pixel position (x, y) on the view plane, object depth is compared by z-values.



At view-plane position (x, y) , surface S_1 has the smallest depth from the view plane and so is visible at that position.

- Three surfaces at varying distance from view plane x_v, y_v , the projection along (x, y) surface S_1 is closest to the view-plane so surface intensity value of S_1 at (x, y) is saved.



► Z-Buffer Algorithm (Depth-Buffer):

❖ **Case 1: Sitting +Z, looking towards Origin -Z**

- Larger Z value = Closer
- Smaller Z value = Farther
- 1. $D(x,y) = 0$ (Initially 0)
 $R(x,y) = I_{\text{background}}$
- 2. $Z_{\text{new}} > D(x,y)$
 $D(x,y) = Z_{\text{new}}$
 $R(x,y) = I_{\text{new}}$

❖ **Case 2: Sitting in -Z & looking towards Origin +Z**

- Smaller Z value = Closer
- Larger Z value = Farther
- 1. $D(x,y) = Z_{\text{max}} (\infty)$
 $R(x,y) = I_{\text{background}}$
- 2. $Z_{\text{new}} < D(x,y)$
 $D(x,y) = Z_{\text{new}}$
 $R(x,y) = I_{\text{new}}$

► Z-Buffer Algorithm (Depth-Buffer):

In the figure, at view plane position (x, y) , surface S_1 has the smallest depth from the view plane, so it is visible at that position.

Depth value for a surface position (x, y) are calculated from the plane equation as;

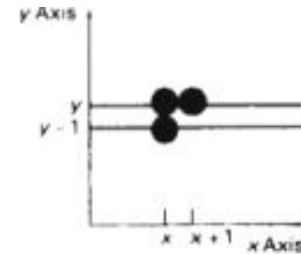
$$z = \frac{-Ax - By - D}{C}$$

Let Depth Z' at position $(x+1, y)$

$$Z' = \frac{-A(x+1) - By - D}{C}$$

$$\Rightarrow Z' = Z - \frac{A}{C} \quad (1)$$

$-\frac{A}{C}$ is constant for each surface so succeeding depth value across a scan line are obtained from preceding values by simple calculation.



► Z-Buffer Algorithm (Depth-Buffer):

Algorithm:

- 1) *Set the buffer values:*
 $\text{DepthBuffer}(x, y) = 0,$
 $\text{FrameBuffer}(x, y) = \text{Background color}$
- 2) *Process each polygon, one at a time as follows:*
 - *For each projected (x, y) pixel position of a polygon, calculate depth 'z'.*
 - *If $z > \text{DepthBuffer}(x, y)$, compute surface color and set*
 $\text{DepthBuffer}(x, y) = z,$
 $\text{FrameBuffer}(x, y) = \text{surfacecolor}(x, y)$

After all surface have been processed, the depth buffer contains depth values for visible surface and frame buffer contains corresponding color values for those surface.

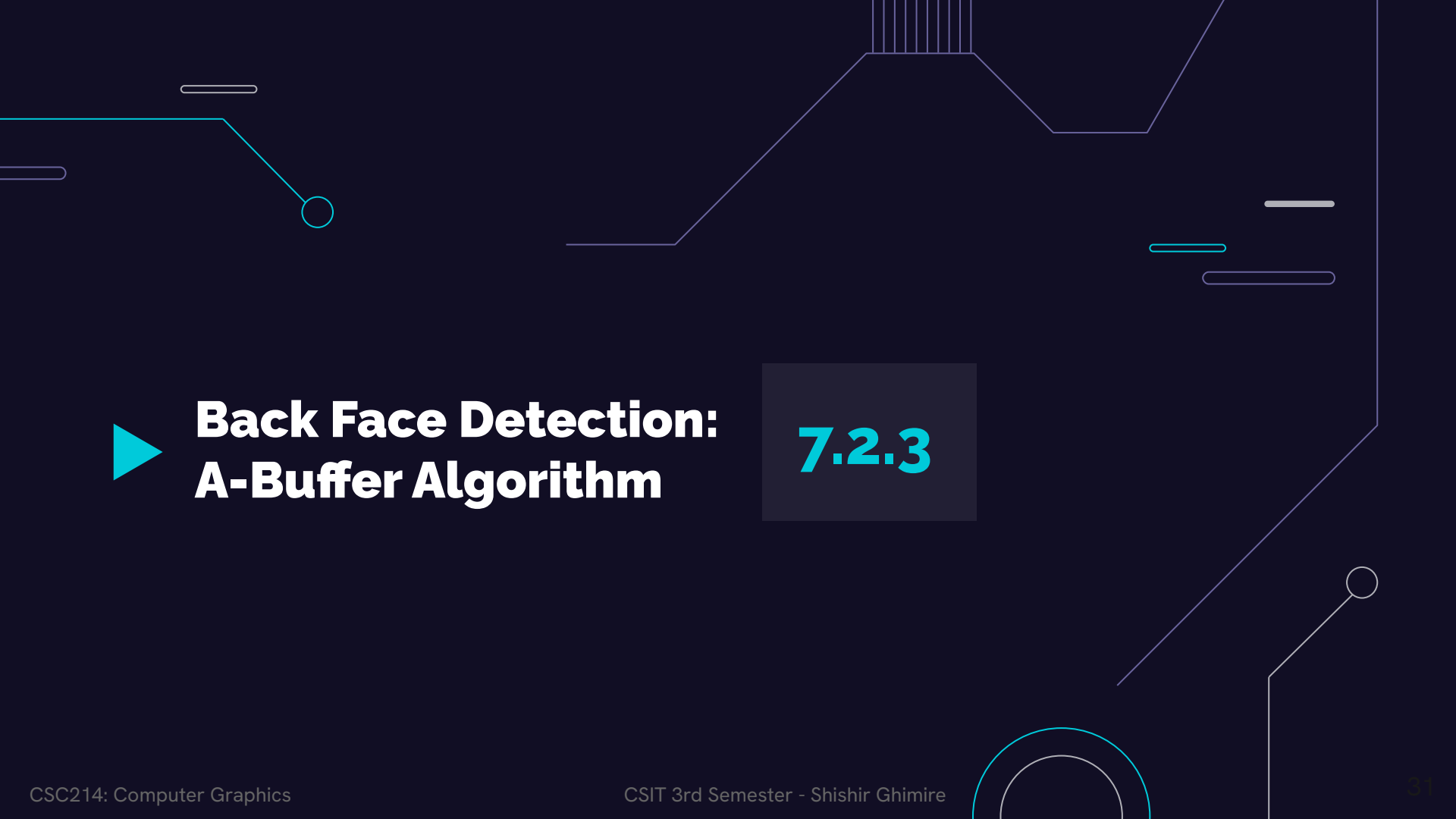
► Z-Buffer Algorithm (Depth-Buffer):

❖ Advantages:

- It is easy to implement.
- It reduces the speed problem if implemented in hardware.
- It processes one object at a time

❖ Disadvantages:

- It requires large memory
- It is a time consuming process as it requires comparison for each pixel instead of for the entire polygon.
- Deals only with opaque object but not for transparent object.



▶ **Back Face Detection: A-Buffer Algorithm**

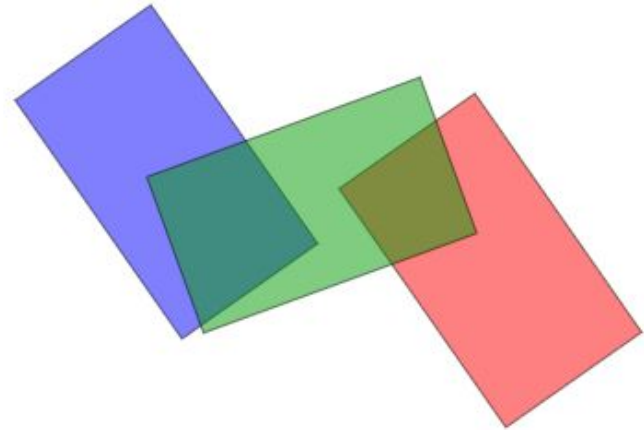
7.2.3

► A- Buffer Algorithm:

- ❖ The A-buffer method is an **extension** of the depth-buffer method.
- ❖ This method is also known as anti-aliased or area-averaged or **accumulation buffer**.
- ❖ The A-buffer method is a visibility detection method developed at **Lucas film studio** for the surface rendering system **REYES** (Render Everything You Ever Saw).
- ❖ The A-buffer expands on the depth buffer method to **allow transparencies**.
- ❖ A **drawback** of Z-buffer method is that it can only find **one visible surface** at each pixel position. That means, it **deals only with opaque surface** and cannot accumulate intensity values for more than one surface, as is **necessary if transparent surface** are to be displayed.
- ❖ It consists of **accumulation(A) buffer** which has two fields:
 - **Depth field:** It stores a **positive or negative** real number.
 - **Intensity field:** It stores **surface data or a pointer value**.

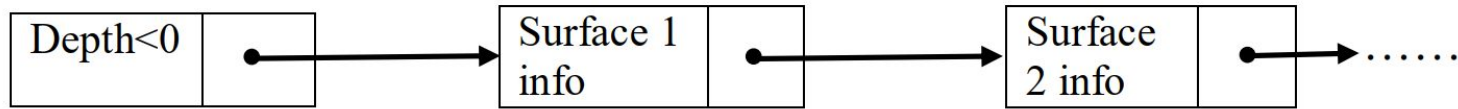
► A-Buffer Algorithm:

- ❖ In A-buffer method, each pixel is made up of a **group of sub-pixels**. The final color of a pixel is computed by **summing up all of its sub-pixels**.
- ❖ Due to this accumulation taking place at sub-pixel level, A-buffer method gets the name **accumulation buffer**.



► A- Buffer Algorithm:

$\text{depth} \geq 0$	RGB and other info
-----------------------	-----------------------

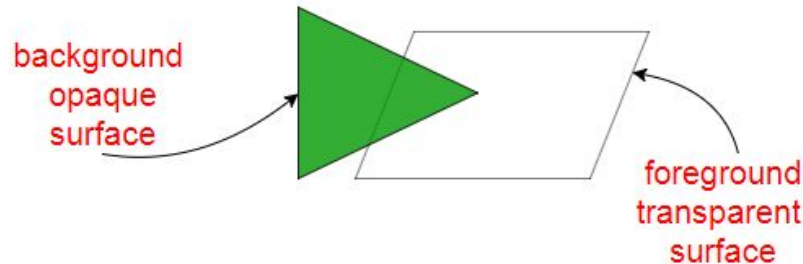


If depth field is non negative (≥ 0), it indicates single surface. The intensity field then stores the RGB components of the surface color at that point and the percent of pixel coverage.

If depth is negative (< 0) it indicates multiple surface. The intensity field then stores a pointer to a linked list of surface data.

► A- Buffer Algorithm:

- ❖ As shown in the above figure, **multiple-surface contributions** to the pixel intensity is indicated by **depth < 0**. The 2nd field, i.e, the intensity field then stores a pointer to a linked list of surface data.
- ❖ A buffer method is **slightly costly** than Z-buffer method because it requires more memory in comparison to the Z-buffer method. It **proceeds just like the depth buffer algorithm**. Here, the depth and opacity are used to determine the final color of the pixel. As shown in the figure below, the A buffer method can be used to show the transparent objects.



► A- Buffer Algorithm:

❖ Surface information in **A-buffer includes:**

- Depth
- Surface identifier
- Opacity parameter
- Percentage of area coverage
- RGB intensity component
- Pointer to the next surface

❖ **Advantage:**

- It provides anti-aliasing in addition to what Z-buffer does.

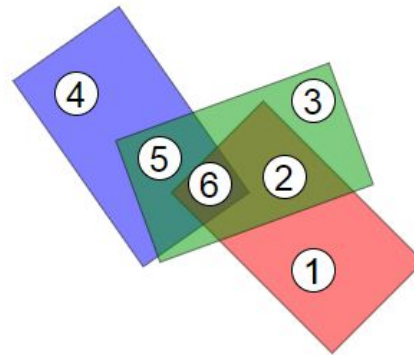
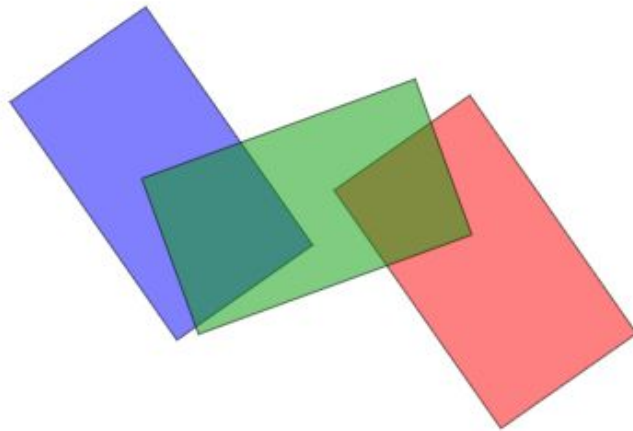
❖ **Disadvantage:**

- It is slightly costly than Z-buffer method because it requires more memory in comparison to the Z-buffer method.

► A- Buffer Algorithm:

❖ Advantage:

- It provides **anti-aliasing** in addition to what Z-buffer does. The usage of A-buffer algorithm for the transparent surfaces is as shown below :



- ① Red
- ② Red → Green
- ③ Green
- ④ Blue
- ⑤ Green → Blue
- ⑥ Red → Green → Blue



▶ Scan Line Algorithm

7.2.4

► Scan Line Algorithm:

It is an **image-space method** for identifying visible surface (removing hidden surface) which is **extension** of the scan line polygon filling for polygon interiors.

- In this method, as each scan line is processed, all polygon surfaces **intersecting that line** are examined to determine which is visible.
- Across each scan line, it **computes and compares** depth values for each **overlapping surface** to determine which is nearest to view plane.
- When the **visible surface** is determined, the intensity value for that position is entered into the **frame buffer**.
- As each scan line is processed all **polygon surfaces** intersecting that line are examined to determine which one is visible.

► Scan Line Algorithm ► Characteristics:

- ❖ It is an **image-space method** for identifying visible surfaces. It deals with **multiple** polygon.
- ❖ It solves the hidden surface problem **one scan line at a time**. (Processing scan line starting from top to bottom of the display).
- ❖ This method calculates Z-value **for only the overlapping surface** which is tested by scan line.
- ❖ Each scan line is processed i.e all polygons surfaces intersecting that line are examined to determine which are visible.
- ❖ Across each scan line , **depth calculations** are made for each overlapping for each overlapping surface to **determine which surface is nearest** to the view.
- ❖ When the visible surface has been determined, the intensity values for that position is entered into the buffer.
- ❖ It requires **following tables: edge table, polygon, active edge list and flag.**

► Scan Line Algorithm ► Components:

❖ Edge Table:

- It contains coordinate **end-points** for each scan line, pointers into the polygon table to identify the surface bounded by each line.

❖ Polygon Table:

- It contains **coefficients of plane equations** for each surface, pointers into edge table, intensity informations of the surfaces.

❖ Active Edge List:

- It contains **edges that cross** the current scan line, shorted in order of increasing x.

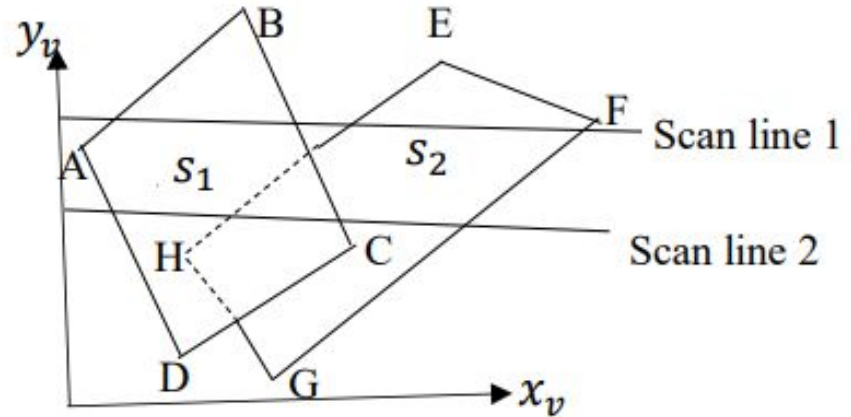
❖ Flag:

- Flag is defined for each surface that is set "ON" or "OFF" to indicate whether a scan line is **inside or outside** of the surface.
- At the LEFTMOST boundary surface flag is "ON" and at right most boundary flag is "OFF".

► Scan Line Algorithm:

The active list for scan line 1 contains edges AB, BC, EH & FG. Similarly for scan line 2, active list contains edges AD, EH, BC & FG.

	Edge pairs	Flags for	
		S_1	S_2
For scan line 1	AB & BC	on	off
	BC & EH	off	off
	EH & FG	off	on
For scan line 2	AD & EH	on	off
	EH & BC	on	on
	BC & FG	off	on



Here, between EH & BC, the flags for both surfaces are ON so depth calculation is necessary. But between other edge pairs no depth calculation is necessary.

► Scan Line Algorithm:

1. Start
2. Established data structure.
 - a. Edge table with endpoints.
 - b. Polygon table with plane coefficients, color and pointer to the edge table.
 - c. Active edge list sorted in increasing order of x.
 - d. A flag for each surface.
3. Repeat for each scan line.
 - a. Update AEL for each scan line y.
 - b. When flag is ON, enter intensity of that surface to refresh buffer.
 - c. When two or more flags are ON, calculate depth and store the intensity of the surface nearest to the view plane.

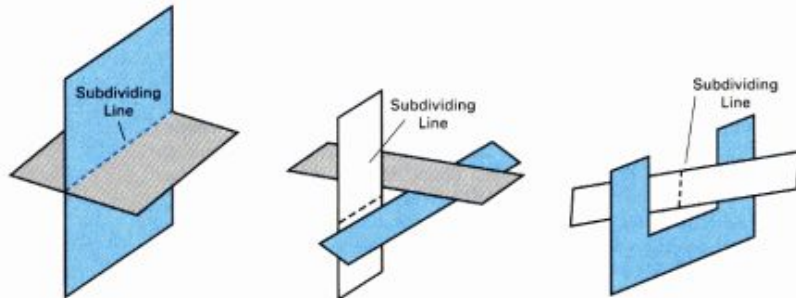
► Scan Line Algorithm:

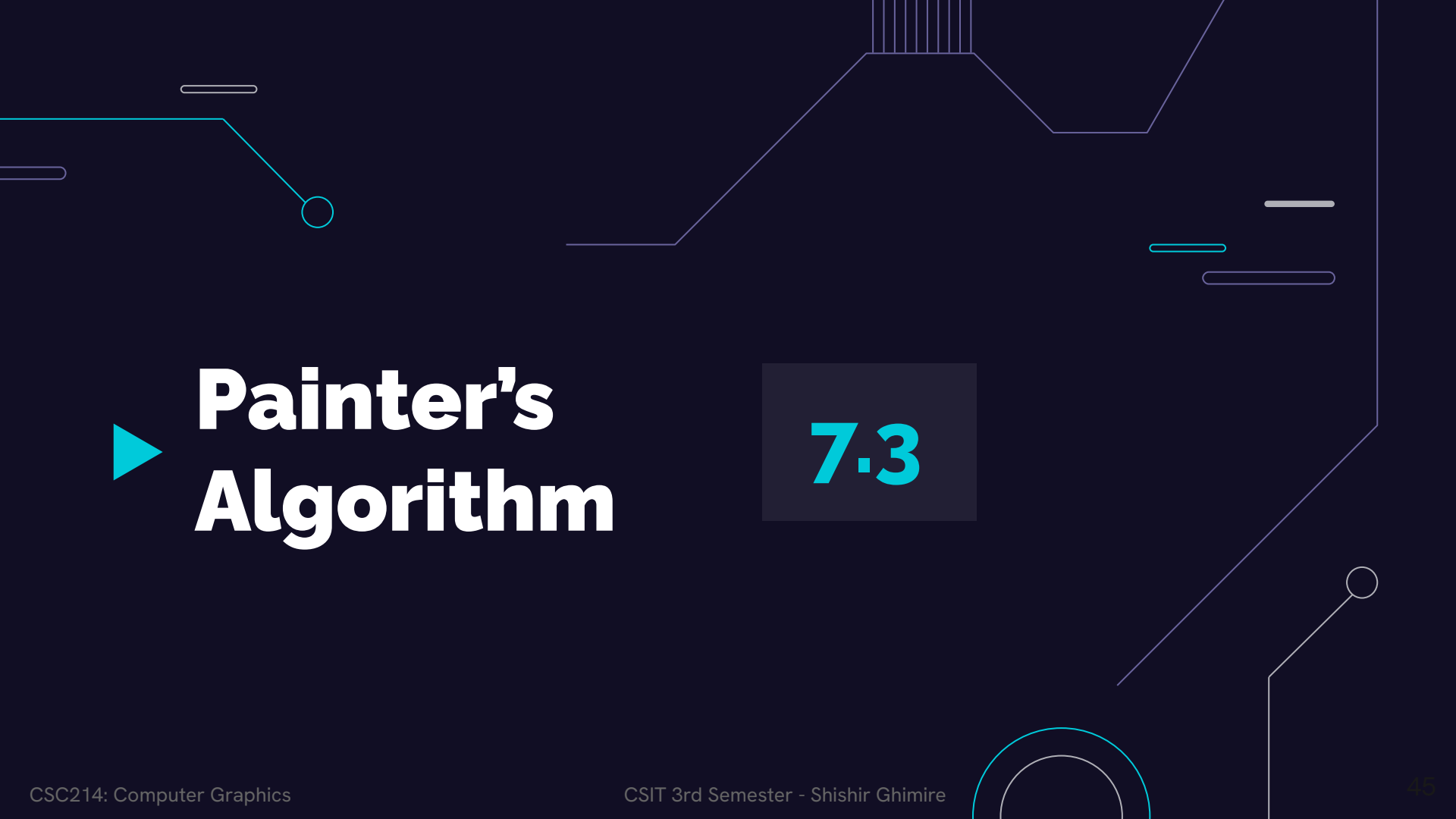
❖ Advantage:

- Any number of overlapping polygon surfaces can be processed with this method.
- Deals with transparent, translucent and opaque surfaces.
- Can be applied to non-polynomial objects.

❖ Limitation:

- The scan line method runs into trouble when surface cut through each other or otherwise cyclically overlap. Such surface need to be divided.
- Additional memory buffer is required.

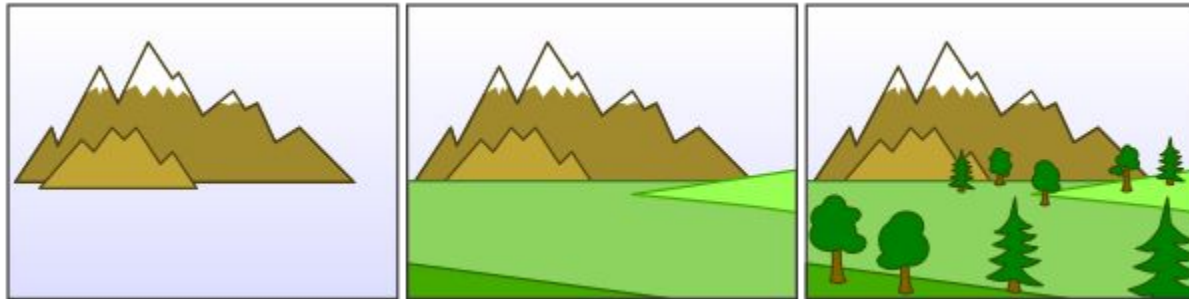
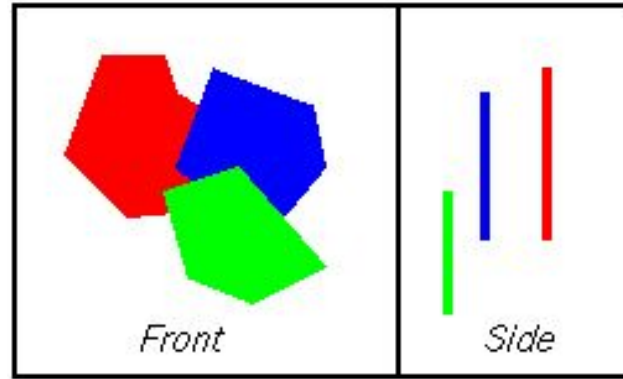


The background features a dark blue field with various white and light blue geometric elements. These include thin lines, some forming a grid-like pattern at the top, and others forming abstract shapes. There are also small circles and rectangles scattered throughout. A prominent light blue triangle points towards the main title.

▶ Painter's Algorithm

7.3

► Painter's Algorithm:



► Painter's Algorithm:

- Also known as **Depth-Sorting Algorithm**.
- A visible surface detection method that uses **both image-space and object-space operations**.
- In this method the surface representation of 3D object are **sorted** in of **decreasing depth from viewer**. Then sorted surface are **scan converted** in order starting with surface of greatest depth for the viewer.
- This algorithm is also called "**Painter's Algorithm**" as it simulates how a painter typically produces his painting by starting with the **background** and then progressively adding new (nearer) objects to the canvas.

► Painter's Algorithm:

The conceptual **steps** that performed in depth-sort algorithm are:

1. Surfaces are sorted in order of **decreasing depth** (z-coordinate).
2. Resolve any ambiguity this may cause when the polygons z-extents **overlap**, splitting polygons if necessary. (The surface 'S' at the end of the list is then **compared** against all other surface to see if there are **any depth overlap**.)
3. Surfaces are scan converted **in order**, starting with the surface of greatest depth.

► Painter's Algorithm:

- ❖ The intensity value of the **farthest surface** are then entered into refresh buffer.
 - i.e Farthest polygon is **displayed first**, then second farthest, and so on finally to the closest polygon.

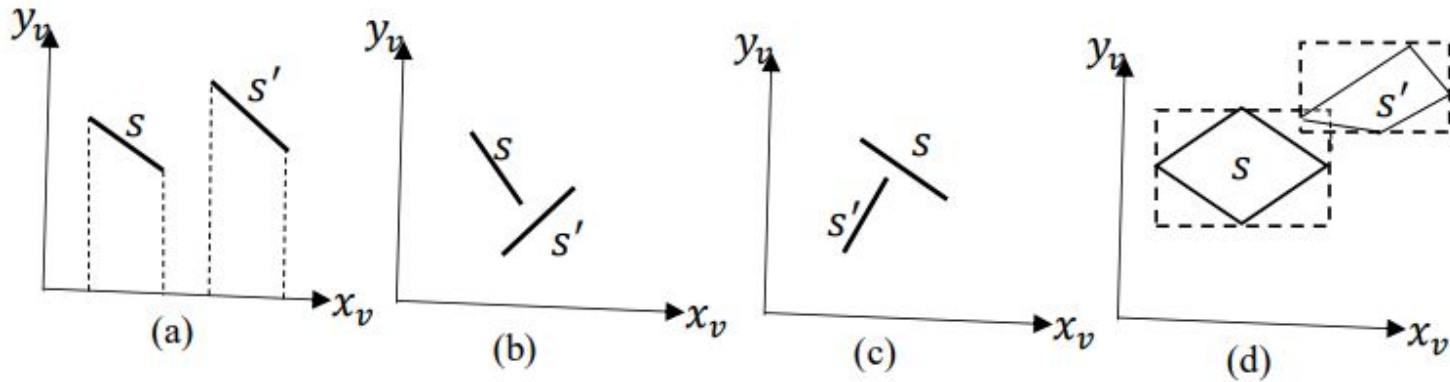
Farthest → Second Farthest → → Closest Polygon

- ❖ After all surface have been processed, the refresh buffer stores the **final intensity values** for all visible surfaces.
- ❖ For only few objects in scene, this method is **very fast** but as the number of objects increases the **sorting process becomes time consuming** and complex.

► Painter's Algorithm:

When there is **depth overlap** with 'S' we make the following tests:

- a) The bounding rectangle for the two surfaces do not overlap
- b) **Surface S** is completely behind the **overlapping surface S'** relative to viewing position.
- c) The **overlapping surface S'** is completely in front of **Surface S** relative to viewing position.
- d) The boundary edge projection of the two surfaces do not overlap.



If any of this test is true, no reordering is necessary. But if all four tests fail then we have to interchange surface S and S' in sorted list.

► Painter's Algorithm:

Advantages:

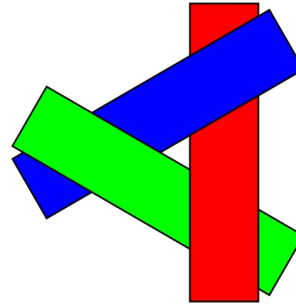
- ❖ The sorting computation is **very fast, so a quick calculation** of image display is possible.
- ❖ The finished image date **remains object based(sorted polygons)**, and can be edited.
- ❖ It can also print at the **highest resolution** available.

Disadvantages:

- ❖ If two or more surfaces alternately obscure each other it get into an **infinite loop**.

► Painter's Algorithm ► Disadvantages:

Problem: One of the major problems in this algorithm is intersecting polygon surfaces. As shown in fig. below.



- Different polygons may have same depth.
- The nearest polygon could also be farthest.
- We cannot use simple depth-sorting to remove the hidden-surfaces in the images.

Solution: For intersecting polygons, we can split one polygon into two or more polygons which can then be painted from back to front. This needs more time to compute intersection between polygons. So it becomes complex algorithm for such surface existence.

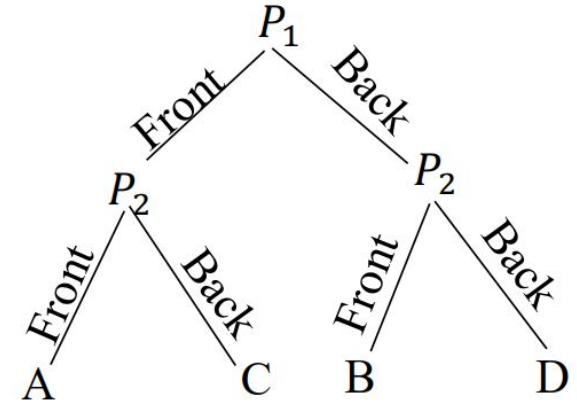
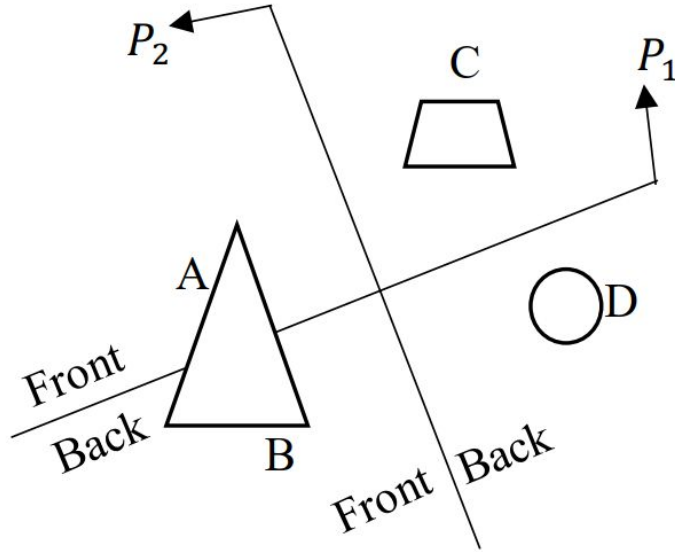
► **BSP Tree Method,
Octree, Ray Tracing**

7.4

► BSP Tree Method:

- ❖ Binary Space Partitioning (BSP) tree is an efficient method for **determining visibility when view reference point changes** but the scene are at fixed position.
 - It involves identifying surfaces that are inside/front and outside/back w.r.t. the partitioning plane at each step of the space division, **relative to the viewing direction**.
 - Start with any plane and find one set of objects behind and the rest in the front.
 - In the tree, the objects are represented as **terminal nodes** with front objects as left branches and back objects as right branches.
- ❖ This algorithm partitions the space into sub divisions until all object list contain no more than one object.

► BSP Tree Method:



BSP tree

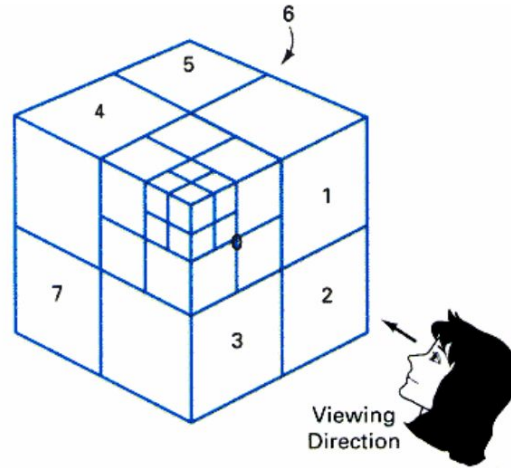
► **BSP Tree Method:**

❖ Disadvantages:

- More polygon splitting may occur than in painter's algorithm.
- Appropriate partitioning hyperplane selection is quite complicated and difficult.

► Octree Method:

- ❖ In this representation **octree nodes** are **projected** onto the viewing surface in a **front-to-back order**. Any surfaces toward the rear of the front octants (0, 1, 2, 3) or in the **back octants (4, 5, 6, 7)** may be **hidden** by the front surfaces.



Octants in Space

► Octree Method:

- ❖ With the numbering method (0, 1, 2, 3, 4, 5, 6, 7), nodes representing octants 0, 1, 2, 3 for the entire region **are visited before** the nodes representing octants 4, 5, 6, 7. Similarly the nodes for the **front four sub-octants of octant 0** are visited before the nodes for the four back suboctants.
- ❖ When a **color is encountered** in an octree node, **the corresponding pixel** in the frame buffer is painted **only if** no previous color has been loaded into the same pixel position.
- ❖ In most cases, both a front and a back octant must be considered in determining the correct color values for a quadrant. But
 - If the **front octant** is homogeneously filled with **some color**, we **do not process** the back octant.
 - If the **front is empty**, it is necessary **only to process the rear octant**.
 - If the front octant has heterogeneous regions, it has to be **subdivided and the suboctants are handled recursively**.

► Ray-Casting Method:

- ❖ Ray tracing also known as **ray casting** is efficient method for visibility detection in the objects.
- ❖ In this method, **visibility of surface can be determined by tracing a ray of light from the center of projection** (viewer's eye) **to objects in the scene**.
 - Find out **which objects** the ray of light intersects.
 - Then, determine **which one** of these **objects is closer** to the viewer.
 - Then, **set the pixel color** to the object.
- ❖ The ray tracing approach is an effective visibility detection method for scenes with **curved surfaces, particularly spheres**.

► Ray-Casting Method:

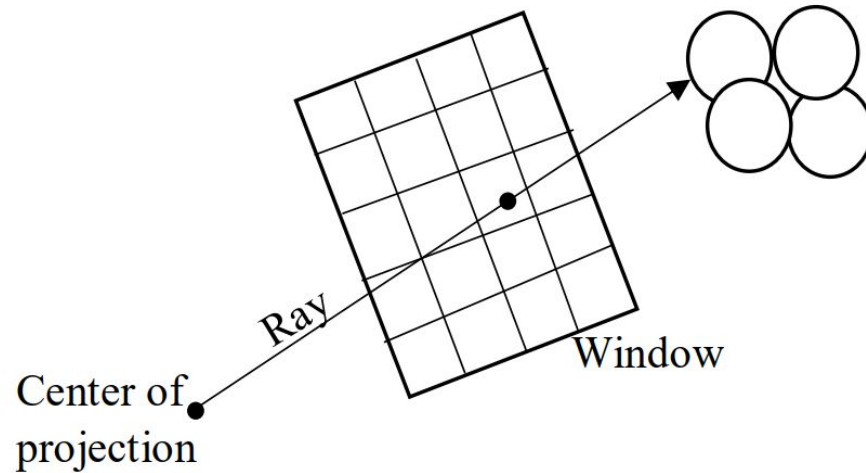
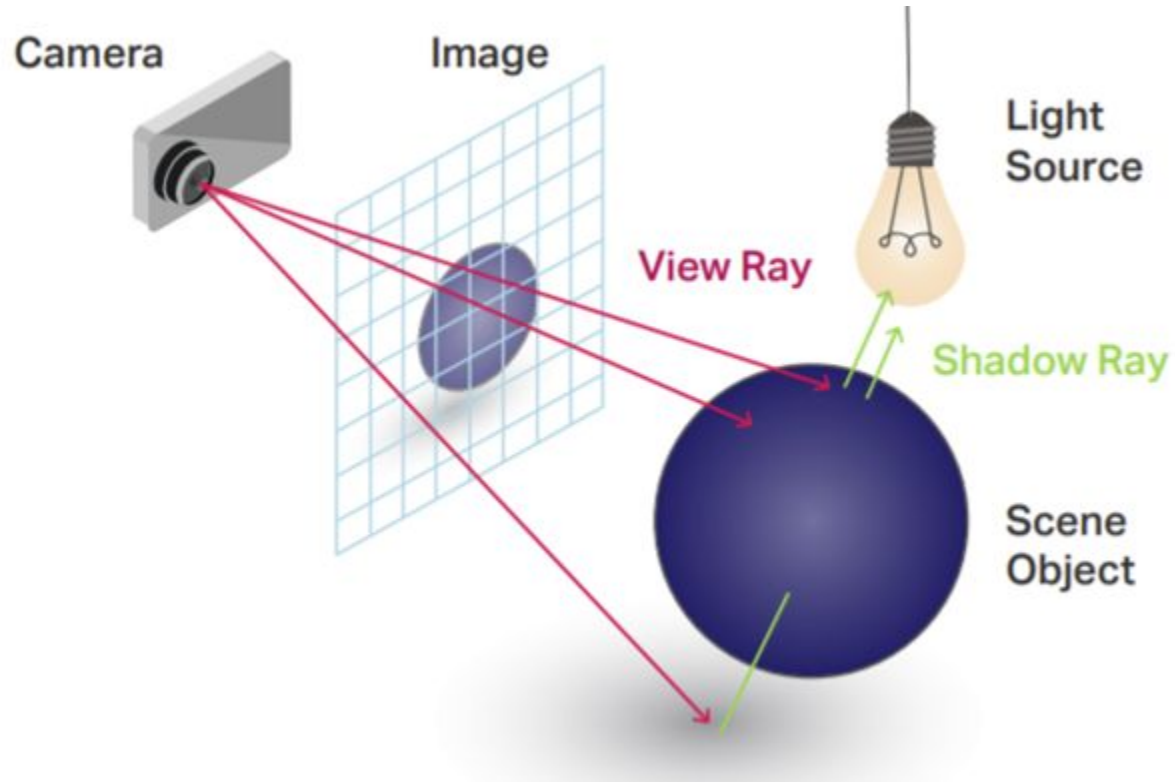


Fig: A ray is fired from the center of projection through each pixel to which the window maps, to determine the closest object intersected.

► Ray-Casting Method:



► Ray-Casting Method:



► Ray-Casting Method:

❖ Characteristics:

- **Trace the path** of an imaginary ray from the viewing position (eye) through **viewing plane** to object in the scene.
- Identify the visible surface by determining which **surface is intersected first by the ray**.
- Can be **easily combined with lightning algorithms** to generate shadow and reflection.
- It is good for curved surface but **too slow** for real time application.

❖ Advantages:

- Conceptually Simple.
- Automatic hidden surface removal (always finds the closest intersection)

► Assignment-7:

Deadline: 23rd February, 2026

1. Why do we require hidden surface removal method? Difference between Image Space Method and Object Space Method.
2. Explain scan line algorithm for Hidden Surface Removal.
3. Explain Depth Buffer Algorithm for Hidden Surface Removal. List its advantages and disadvantages.
4. What do you mean by hidden surface removal? Describe any hidden surface removal algorithm with example.
5. What is the min-max test used in Z-Buffer Method? When does min-max fails?
6. Explain painter's algorithm. (Depth-sorting Algorithm)
7. What is the role of ray tracing in visible surface detection? Explain How scan line algorithm is used for back face detection.



▶ **THANKS!**

Do you have any questions?

theciceerguy@p_m.me

9841893035

ghimires.com.np